[ПОЛНОЕ НАИМЕНОВАНИЕ МИНИСТЕРСТВА]
[ПОЛНОЕ НАИМЕНОВАНИЕ ОБРАЗОВАТЕЛЬНОЙ ОРГАНИЗАЦИИ]
[ИНСТИТУТ ИЛИ ФАКУЛЬТЕТ]
[КАФЕДРА]

КУРСОВАЯ РАБОТА

по дисциплине
«Алгоритмические основы обработки данных»

на тему
«Разработка программы для проведения конкурса»

Выполнил:
студент группы *[ГРУППА]*
[ФИО СТУДЕНТА]

Руководитель:
[ДОЛЖНОСТЬ, УЧЁНАЯ СТЕПЕНЬ]
[ФИО ПРЕПОДАВАТЕЛЯ]

[ГОРОД]
2026

Содержание

ВВЕДЕНИЕ 4

1 Техническое задание 6
1.1 Введение 6
1.2 Основание для разработки 6
1.3 Назначение разработки 6
1.4 Требования к программе 7
1.5 Требования к программной документации 8
1.6 Этапы разработки 9

2 Обзор способов организации данных 10
2.1 Массив фиксированного размера 10
2.2 Связный список 10
2.3 Ассоциативные контейнеры 10
2.4 Динамический массив 10
2.5 Представление необязательного балла 11
2.6 Оценка сложности 11

3 Описание программы 12
3.1 Общие сведения 12
3.2 Архитектура и состав модулей 12
3.3 Модель данных 12
3.4 Описание основных алгоритмов 13
3.5 Файловое хранение 14
3.6 Пользовательский интерфейс 15
3.7 Сборка и запуск 15

4 Тестирование программы 16
4.1 Методика тестирования 16
4.2 Контрольные примеры 16
4.3 Анализ результатов 17
4.4 Ограничения текущей версии 17

ЗАКЛЮЧЕНИЕ 18

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 19

ПРИЛОЖЕНИЕ А 20

ВВЕДЕНИЕ

Автоматизация проведения конкурсов позволяет исключить ошибки ручного учёта, обеспечить однозначность результатов и ускорить формирование итогового рейтинга. Для решения этой задачи требуется программа, которая контролирует этапы конкурса, хранит сведения об участниках, проверяет корректность вводимых данных, регистрирует баллы и определяет призёров по заданным правилам.

Цель курсовой работы заключается в разработке консольной программы на языке C++, предназначенной для регистрации участников и результатов конкурса, сортировки итогового списка и сохранения данных в текстовых файлах.

Для достижения поставленной цели необходимо решить следующие задачи:

1. проанализировать предметную область и сформулировать требования к программе;
2. выбрать структуры данных и алгоритмы, соответствующие ограничениям задачи;
3. спроектировать объектную модель и разделить программу на логические модули;
4. реализовать проверку входных данных и управление состояниями конкурса;
5. организовать сохранение незавершённой регистрации между сеансами;
6. реализовать сортировку результатов и правило расширения числа призёров при равенстве баллов;
7. разработать автоматические тесты и провести функциональную проверку программы.

Объектом исследования является процесс программной обработки данных о конкурсе. Предмет исследования — методы организации данных, алгоритмы сортировки и средства объектно-ориентированного программирования C++.

Практическая значимость работы состоит в создании законченного консольного приложения, которое может применяться как основа для учебных, творческих и иных конкурсов с балльной системой оценки. Архитектура про-

граммы допускает дальнейшее развитие, в частности добавление графического интерфейса или иной системы хранения данных.

1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ

1.1 Введение

Техническое задание определяет назначение, функции, ограничения и условия эксплуатации программы «Конкурс». Программа разрабатывается в рамках курсовой работы по дисциплине «Алгоритмические основы обработки данных».

1.1.1 Наименование программы

Полное наименование: «Программа для проведения конкурса». Краткое наименование: «Конкурс». Имя исполняемого файла при стандартной сборке CMake: `contest` в Unix-подобных системах и `contest.exe` в Windows.

1.1.2 Область применения

Программа предназначена для небольших и средних конкурсов, в которых каждому зарегистрированному участнику присваивается один итоговый балл. Применение возможно в образовательных организациях, кружках, секциях и при проведении внутренних соревнований.

1.2 Основание для разработки

Основанием для разработки является задание на курсовую работу. Реализация должна демонстрировать владение языком C++, принципами объектно-ориентированного проектирования, стандартными контейнерами и алгоритмами, файловым вводом-выводом, обработкой ошибочных ситуаций и автоматическим тестированием.

1.3 Назначение разработки

Функциональное назначение программы — поддержка полного цикла конкурса:

– открытие конкурса и указание количества призовых мест;

– регистрация участников;

– завершение регистрации;

– ввод итогового балла каждого участника;

– завершение конкурса;

– сортировка участников по убыванию баллов;

– вывод результатов на экран и в текстовый файл.

Эксплуатационное назначение — предоставить оператору простой консольный интерфейс, не требующий специальных навыков или установки дополнительных графических библиотек.

1.4 Требования к программе

1.4.1 Функциональные требования

Программа должна поддерживать не более 1000 участников и от 1 до 10 призовых мест. Участник описывается уникальным положительным целочисленным идентификатором и непустым именем. Для каждого участника разрешается зарегистрировать ровно один итоговый балл в диапазоне от 0 до 100 включительно.

Регистрация может выполняться за несколько сеансов. При штатном выходе на этапе регистрации актуальный список сохраняется в файле `participants.`. При следующем запуске записи загружаются автоматически. После завершения регистрации временный файл удаляется, поскольку ввод баллов и формирование итогов выполняются в одном сеансе.

Для завершения регистрации количество участников должно быть не меньше заявленного числа призовых мест. Завершение конкурса допускается только после ввода балла для каждого участника.

Итоговый список сортируется по убыванию балла. Для детерминированного порядка при равных баллах дополнительно используются имя, а затем идентификатор. Если несколько участников имеют такой же балл, как участник на последней призовой позиции, все они признаются призёрами. Например, при двух призовых местах и баллах 100, 90, 90, 70 призёрами становятся первые три участника.

1.4.2 Требования к надёжности

Программа должна отклонять:

– неположительный или повторяющийся идентификатор;

– пустое имя и имя с символами табуляции либо перевода строки;

– балл вне диапазона от 0 до 100;

– повторный ввод балла;

– завершение регистрации при недостаточном числе участников;

– завершение конкурса при наличии участника без балла.

Запись списка участников выполняется через временный файл. После успешного закрытия потока временный файл переименовывается в основной. Такой порядок уменьшает вероятность повреждения уже сохранённых данных при ошибке записи.

1.4.3 Требования к техническим и программным средствам

Минимальные требования: персональный компьютер, операционная система Windows 10 или новее, Linux либо macOS; компилятор с поддержкой C++17; CMake версии 3.16 или новее. Для сборки в Windows может применяться Microsoft Visual Studio с компонентами разработки на C++.

1.4.4 Требования к входным и выходным данным

Входные данные вводятся через стандартный поток ввода и читаются из файла `participants.txt`. Файл незавершённой регистрации содержит по одной записи в строке: идентификатор, символ табуляции и имя участника.

Выходные данные отображаются в терминале. После завершения конкурса формируется файл `results.txt`. Он содержит место, идентификатор, имя, балл и статус призёра для каждого участника.

1.5 Требования к программной документации

В состав программной документации входят:

– исходные тексты программы;

– файл конфигурации сборки `CMakeLists.txt`;

– инструкция по сборке и запуску `README.md`;

– набор автоматических тестов;

– настоящий отчёт.

1.6 Этапы разработки

Разработка выполнена по следующим этапам: анализ требований; выбор структур данных; проектирование классов и модулей; реализация; модульное тестирование; проверка полного пользовательского сценария; подготовка документации.

2 ОБЗОР СПОСОБОВ ОРГАНИЗАЦИИ ДАННЫХ

2.1 Массив фиксированного размера

Массив обеспечивает постоянное время доступа по индексу и не требует динамического изменения структуры. Однако фактическое число участников заранее неизвестно. Использование массива на 1000 элементов привело бы к необходимости отдельно хранить текущий размер и усложнило бы работу с объектами, содержащими строки.

2.2 Связный список

Связный список обеспечивает вставку элемента без перемещения остальных записей. Для данной задачи вставка выполняется только в конец, а основная итоговая операция — сортировка. Раздельное размещение узлов в памяти ухудшает локальность данных, а произвольный доступ имеет линейную сложность.

2.3 Ассоциативные контейнеры

Контейнеры `std::map` и `std::unordered_map` удобны для быстрого поиска по идентификатору. Однако итоговая сортировка всё равно потребовала бы переноса записей в последовательный контейнер. При максимуме 1000 элементов линейный поиск остаётся приемлемым, а единый контейнер делает реализацию проще.

2.4 Динамический массив

Контейнер `std::vector` хранит элементы последовательно, автоматически управляет памятью, поддерживает добавление в конец и совместим со стандартными алгоритмами C++. Сортировка копии вектора может быть выполнена функцией `std::sort`. Поэтому `std::vector<Participant>` выбран основной структурой хранения.

2.5 Представление необязательного балла

До этапа оценивания балл участника отсутствует. Использование специального числового значения, например -1, смешивало бы состояние «балл не введён» с предметным значением. Тип `std::optional<int>` явно отражает наличие или отсутствие результата и позволяет компилятору контролировать обращения к значению.

2.6 Оценка сложности

Пусть n — число участников, $n \leq 1000$. Проверка уникальности идентификатора и поиск участника выполняются линейно за $O(n)$. Регистрация всех участников и всех баллов в худшем случае требует $O(n^2)$ операций сравнения. При заданном ограничении это не более порядка одного миллиона сравнений, что не создаёт заметной задержки.

Итоговая сортировка выполняется за $O(n \log n)$, а построение списка призёров — за $O(n)$. Память, занимаемая основным контейнером и его сортированной копией, имеет порядок $O(n)$.

Таблица 1 – Сравнение рассмотренных структур данных

Структура	**Добавление**	**Сортировка**	**Вывод**
Фиксированный массив	простое, но нужна ручная граница	возможна на месте	не соответствует переменному размеру
Связный список	эффективно	менее удобна для задачи	лишние узлы и указатели
Ассоциативный контейнер	быстрый поиск по ключу	нужен дополнительный порядок по баллу	избыточен при $n \leq 1000$
`std::vector`	амортизированно $O(1)$ в конец	`std::sort`, $O(n \log n)$	выбран для реализации

3 ОПИСАНИЕ ПРОГРАММЫ

3.1 Общие сведения

Программа написана на C++17 и собирается системой CMake. Проект разделён на библиотеку предметной логики `contest_core`, исполняемое приложение `contest` и тестовый исполняемый файл `contest_tests`. Такое разделение позволяет проверять правила конкурса без имитации интерактивного ввода.

3.2 Архитектура и состав модулей

Структура исходных файлов представлена в таблице 2.

Таблица 2 – Назначение файлов проекта

Файл	**Назначение**
`include/Participant.h`	Структура участника: идентификатор, имя и необязательный балл.
`include/Contest.h`	Интерфейс класса конкурса, состояния и ограничения.
`src/Contest.cpp`	Реализация бизнес-правил, сортировки и определения призёров.
`include/Storage.h`	Интерфейс функций файлового хранения.
`src/Storage.cpp`	Загрузка и сохранение участников, экспорт итогов.
`src/main.cpp`	Проверка консольного ввода, меню и координация этапов.
`tests/contest_tests.cpp`	Автоматические тесты основных правил.
`CMakeLists.txt`	Описание целей сборки и регистрации теста CTest.

3.3 Модель данных

Структура `Participant` содержит поле `id` типа `int`, поле `name` типа `std::string` и поле `score` типа `std::optional<int>`.

Класс `Contest` инкапсулирует число призовых мест, состояние конкур-

са и вектор участников. Внешний код получает к данным доступ только через методы класса. Ошибочные операции возвращают `false` и записывают понятное сообщение в строковый параметр `error`.

Состояние задаётся перечислением:

– `Registration` — разрешено добавление участников;

– `Scoring` — регистрация закрыта, разрешён ввод баллов;

– `Finished` — конкурс завершён, результаты окончательны.

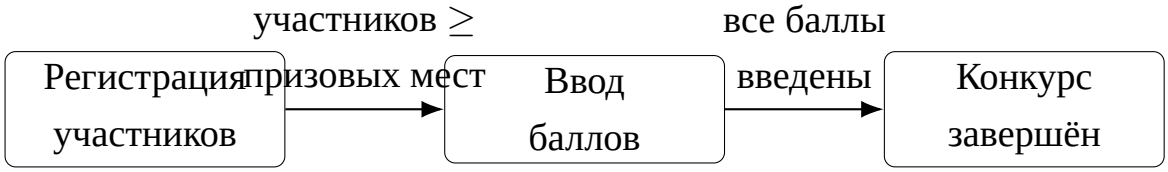


Рис. 1 – Состояния конкурса

3.4 Описание основных алгоритмов

3.4.1 Регистрация участника

Метод `registerParticipant` проверяет текущее состояние конкурса, ограничение на размер вектора, положительность идентификатора, допустимость имени и отсутствие участника с таким же идентификатором. После успешной проверки объект добавляется в конец вектора с отсутствующим значением балла.

3.4.2 Завершение регистрации

Метод `finishRegistration` сравнивает число участников с количеством призовых мест. Если данных достаточно, состояние меняется на `Scoring`. В интерфейсном модуле после этого удаляется файл незавершённой регистрации.

3.4.3 Регистрация балла

Метод `registerScore` доступен только в состоянии `Scoring`. Балл проверяется на принадлежность диапазону $[0; 100]$, затем участник ищется по идентификатору. Значение присваивается только при отсутствии ранее введённого результата.

3.4.4 Сортировка результатов

Метод `standings` создаёт копию вектора и вызывает `std::sort` с компаратором. Сначала сравниваются баллы в порядке убывания, затем имена в лексикографическом порядке, затем идентификаторы. Дополнительные критерии обеспечивают воспроизводимость результата.

3.4.5 Определение призёров

После сортировки определяется балл участника с индексом $p - 1$, где p — заданное число призовых мест. Этот балл является пороговым. Призёрами считаются все участники, чей результат не меньше порогового:

$$W = \{x \in P \mid score(x) \geq score(P_{p-1})\},$$

где P — отсортированный список, W — множество призёров.

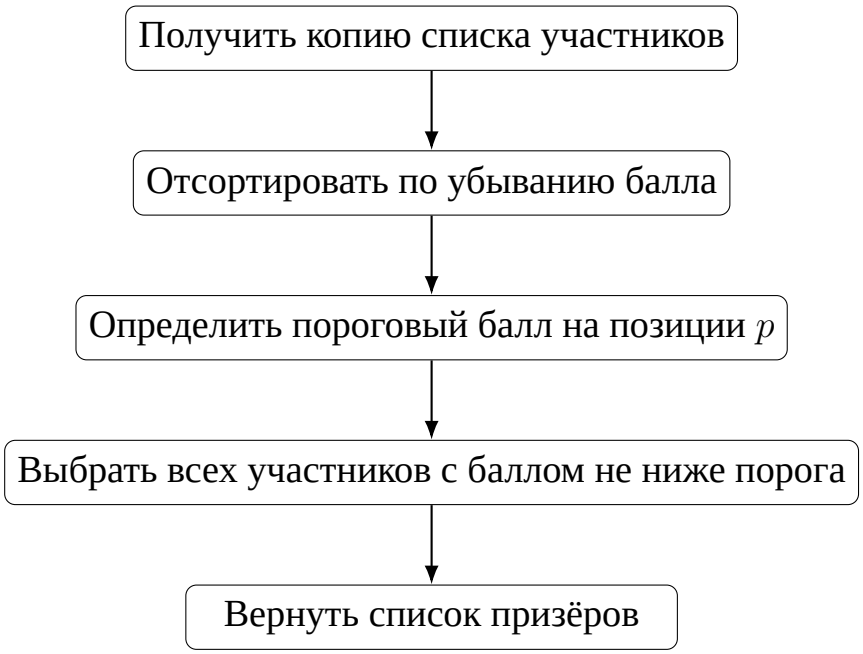


Рис. 2 – Алгоритм определения призёров

3.5 Файловое хранение

Модуль `Storage` не зависит от консольного интерфейса. Функция `loadPart` читает файл построчно и разделяет идентификатор и имя по табуляции. При нарушении формата формируется сообщение с номером проблемной строки.

Функция `saveParticipants` сначала записывает полный актуальный

список во временный файл `participants.txt.tmp`. После проверки потока временный файл атомарно переименовывается, насколько это допускает файловая система. При необходимости существующий целевой файл удаляется перед повторной попыткой переименования.

Функция `saveResults` получает уже отсортированный список, вычисляет пороговый балл и создаёт человекочитаемую таблицу. В файл записываются как призёры, так и остальные участники.

3.6 Пользовательский интерфейс

При запуске оператор вводит число призовых мест. Затем программа автоматически пытается загрузить незавершённую регистрацию. Меню регистрации содержит команды добавления участника, просмотра списка, завершения регистрации и сохранения с выходом.

После закрытия регистрации открывается меню результатов. Оператор вводит идентификатор и балл каждого участника, при необходимости просматривает текущие данные и завершает конкурс. Если не все оценки заполнены, программа сообщает идентификатор первого участника без балла.

3.7 Сборка и запуск

Для конфигурации и сборки проекта используются команды:

```
cmake -S . -B build
cmake --build build --config Release
```

В Windows с многоконфигурационным генератором программа запускается из каталога `build/Release`. В Linux и macOS при одноконфигурационной сборке исполняемый файл обычно расположен непосредственно в каталоге `build`.

Для запуска тестов применяется команда:

```
ctest --test-dir build --output-on-failure
```

4 ТЕСТИРОВАНИЕ ПРОГРАММЫ

4.1 Методика тестирования

Проверка выполнена на двух уровнях. Модульные тесты непосредственно вызывают методы класса `Contest`. Сквозная проверка запускает собранную программу с последовательностью пользовательских команд, проверяет создание файла незавершённой регистрации, повторный запуск, удаление временного файла после закрытия регистрации и наличие участников в итоговом файле.

В тестовой цели используется стандартный макрос `assert`. Тест регистрируется в CTest, поэтому ненулевой код завершения или срабатывание утверждения приводит к фиксации ошибки.

4.2 Контрольные примеры

Таблица 3 – Результаты функциональной проверки

№	Проверяемая ситуация	Ожидаемый результат	Итог
1	ID равен нулю	Регистрация отклонена	пройден
2	Повторяющийся ID	Вторая запись отклонена	пройден
3	Балл 101	Значение отклонено	пройден
4	Повторный ввод балла	Второе значение отклонено	пройден
5	Не всем назначены баллы	Завершение конкурса отклонено	пройден
6	Два места, результаты 100, 90, 90, 70	Три участника признаны призёрами	пройден
7	Выход во время регистрации	Создан файл `participants.txt`	пройден
8	Повторный запуск	Сохранённые участники загружены	пройден
9	Завершение конкурса	Создан упорядоченный `results.txt`	пройден

4.3 Анализ результатов

Автоматический тест `contest_tests` завершился успешно. Сквозной сценарий подтвердил корректность сохранения и возобновления регистрации. В контрольном примере с равенством баллов на призовой границе список победителей расширился до трёх человек, что соответствует согласованному правилу.

Проверка также подтвердила, что предметная логика не допускает переход к следующему этапу при нарушении обязательных условий. Сообщения об ошибках содержат причину отказа и позволяют оператору исправить ввод.

4.4 Ограничения текущей версии

Текущая версия является консольной. Количество призовых мест вводится заново при каждом запуске, поэтому при продолжении сохранённой регистрации оператор должен указать то же значение. Файл результатов хранит текст, ориентированный на чтение человеком, и не предназначен для импорта в базу данных. Регистрация баллов, согласно заданию, не восстанавливается после прерывания сеанса.

В дальнейшем возможно сохранить параметры конкурса вместе со списком участников, добавить журналирование, экспорт в CSV и графический интерфейс. Эти изменения не требуют пересмотра класса `Contest`, поскольку интерфейс и хранение уже вынесены в отдельные модули.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы разработана консольная программа «Конкурс» на языке C++17. Реализованы регистрация до 1000 участников, контроль уникальности идентификаторов, ввод итоговых баллов, управление этапами конкурса, сортировка результатов и формирование текстового отчёта.

Для представления изменяемого набора участников выбран контейнер `std::v` а отсутствие результата до этапа оценивания выражено типом `std::optional`. Предметные правила инкапсулированы классом `Contest`; файловые операции вынесены в модуль `Storage`; взаимодействие с оператором сосредоточено в `main.cpp`. Такое разделение уменьшает связанность компонентов и упрощает тестирование.

Особенностью реализации является корректная обработка равенства баллов на призовой границе. Число призёров расширяется, чтобы все участники с одинаковым пограничным результатом получили призовое место. Незавершённая регистрация сохраняется между сеансами через текстовый файл.

Разработанные автоматические тесты проверяют основные ограничения и переходы состояний. Сборка и тестирование проекта выполнены успешно. Поставленная цель достигнута, а сформированная архитектура позволяет в дальнейшем добавить графический интерфейс без изменения основной логики конкурса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — Москва: Стандартинформ, 2017.
2. ГОСТ Р 7.0.100–2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — Москва: Стандартинформ, 2018.
3. Страуструп Б. Язык программирования C++. Специальное издание. — Москва: Бином, 2016.
4. Липпман С., Лажойе Ж., Му Б. Язык программирования C++. Базовый курс. — Москва: Вильямс, 2017.
5. ISO/IEC 14882:2017. Programming Languages — C++. — Geneva: ISO, 2017.
6. CMake Reference Documentation [Электронный ресурс]. — URL: https://cmake.org/documentation/ (дата обращения: 26.10.2026).
7. cppreference.com. C++ reference [Электронный ресурс]. — URL: https://en.cppreference.com/w/cpp (дата обращения: 26.10.2026).

ПРИЛОЖЕНИЕ А

Листинг основных файлов программы

```
#pragma once

#include <optional>
#include <string>

struct Participant {
    int id{};
    std::string name;
    std::optional<int> score;
};
```

```
#pragma once

#include "Participant.h"

#include <cstddef>
#include <string>
#include <vector>

class Contest {
public:
    enum class State {
        Registration,
        Scoring,
        Finished
    };

    static constexpr std::size_t MaxParticipants = 1000;
    static constexpr int MinPrizePlaces = 1;
    static constexpr int MaxPrizePlaces = 10;

    explicit Contest(int prizePlaces);

    bool registerParticipant(int id, const std::string& name, std::string& error);
    bool finishRegistration(std::string& error);
    bool registerScore(int participantId, int score, std::string& error);
    bool finishContest(std::string& error);

    [[nodiscard]] int prizePlaces() const;
    [[nodiscard]] State state() const;
    [[nodiscard]] const std::vector<Participant>& participants() const;
    [[nodiscard]] std::vector<Participant> standings() const;
    [[nodiscard]] std::vector<Participant> winners() const;

private:
    int prizePlaces_;
    State state_{State::Registration};
    std::vector<Participant> participants_;
};
```

```
#include "Contest.h"

#include <algorithm>
#include <cctype>
#include <stdexcept>

namespace {

bool isBlank(const std::string& text) {
    return std::all_of(text.begin(), text.end(), [](unsigned char ch) {
        return std::isspace(ch) != 0;
    });
}
```

```
}  // namespace

Contest::Contest(int prizePlaces) : prizePlaces_(prizePlaces) {
    if (prizePlaces < MinPrizePlaces || prizePlaces > MaxPrizePlaces) {
        throw std::invalid_argument("Количество призовых мест должно быть от 1 до 10.");
    }
}

bool Contest::registerParticipant(int id, const std::string& name, std::string& error) {
    if (state_ != State::Registration) {
        error = "Регистрация участников уже завершена.";
        return false;
    }
    if (participants_.size() >= MaxParticipants) {
        error = "Достигнуто максимальное количество участников: 1000.";
        return false;
    }
    if (id <= 0) {
        error = "Идентификатор должен быть положительным целым числом.";
        return false;
    }
    if (name.empty() || isBlank(name)) {
        error = "Имя участника не может быть пустым.";
        return false;
    }
    if (name.find('\t') != std::string::npos || name.find('\n') != std::string::npos ||
        name.find('\r') != std::string::npos) {
        error = "Имя не должно содержать символы табуляции или перевода строки.";
        return false;
    }
    const auto duplicate = std::find_if(participants_.begin(), participants_.end(),
                                        [id](const Participant& participant) {
                                            return participant.id == id;
                                        });
    if (duplicate != participants_.end()) {
        error = "Участник с таким идентификатором уже зарегистрирован.";
        return false;
    }

    participants_.push_back({id, name, std::nullopt});
    return true;
}

bool Contest::finishRegistration(std::string& error) {
    if (state_ != State::Registration) {
        error = "Регистрация уже завершена.";
        return false;
    }
    if (participants_.size() < static_cast<std::size_t>(prizePlaces_)) {
        error = "Участников не может быть меньше, чем призовых мест.";
        return false;
    }
    state_ = State::Scoring;
    return true;
}

bool Contest::registerScore(int participantId, int score, std::string& error) {
    if (state_ != State::Scoring) {
        error = "Баллы можно вводить только после окончания регистрации.";
        return false;
    }
    if (score < 0 || score > 100) {
        error = "Балл должен находиться в диапазоне от 0 до 100.";
        return false;
    }

    const auto participant = std::find_if(participants_.begin(), participants_.end(),
                                          [participantId](const Participant& item) {
                                              return item.id == participantId;
                                          });
    if (participant == participants_.end()) {
        error = "Участник с указанным идентификатором не найден.";
        return false;
    }
    if (participant->score.has_value()) {
        error = "Балл этого участника уже зарегистрирован.";
```

```
        return false;
    }

    participant->score = score;
    return true;
}

bool Contest::finishContest(std::string& error) {
    if (state_ != State::Scoring) {
        error = "Сейчас конкурс нельзя завершить.";
        return false;
    }
    const auto missingScore = std::find_if(participants_.begin(), participants_.end(),
                                           [](const Participant& participant) {
                                               return !participant.score.has_value();
                                           });
    if (missingScore != participants_.end()) {
        error = "Не введён балл участника с идентификатором " +
                std::to_string(missingScore->id) + ".";
        return false;
    }
    state_ = State::Finished;
    return true;
}

int Contest::prizePlaces() const {
    return prizePlaces_;
}

Contest::State Contest::state() const {
    return state_;
}

const std::vector<Participant>& Contest::participants() const {
    return participants_;
}

std::vector<Participant> Contest::standings() const {
    auto result = participants_;
    std::sort(result.begin(), result.end(), [](const Participant& left, const Participant& right) {
        if (left.score != right.score) {
            return left.score.value_or(-1) > right.score.value_or(-1);
        }
        if (left.name != right.name) {
            return left.name < right.name;
        }
        return left.id < right.id;
    });
    return result;
}

std::vector<Participant> Contest::winners() const {
    if (state_ != State::Finished) {
        return {};
    }

    const auto sorted = standings();
    const int cutoff = sorted.at(static_cast<std::size_t>(prizePlaces_ - 1)).score.value();
    std::vector<Participant> result;
    std::copy_if(sorted.begin(), sorted.end(), std::back_inserter(result),
                 [cutoff](const Participant& participant) {
                     return participant.score.value() >= cutoff;
                 });
    return result;
}



#pragma once

#include "Participant.h"

#include <filesystem>
#include <string>
#include <vector>

namespace Storage {
```

```
bool loadParticipants(const std::filesystem::path& path,
                      std::vector<Participant>& participants,
                      std::string& error);

bool saveParticipants(const std::filesystem::path& path,
                      const std::vector<Participant>& participants,
                      std::string& error);

bool saveResults(const std::filesystem::path& path,
                 const std::vector<Participant>& standings,
                 int prizePlaces,
                 std::string& error);

}  // namespace Storage


#include "Storage.h"

#include <fstream>
#include <sstream>
#include <system_error>

namespace Storage {

bool loadParticipants(const std::filesystem::path& path,
                      std::vector<Participant>& participants,
                      std::string& error) {
    participants.clear();
    if (!std::filesystem::exists(path)) {
        return true;
    }

    std::ifstream input(path);
    if (!input) {
        error = "Не удалось открыть файл участников: " + path.string();
        return false;
    }

    std::string line;
    std::size_t lineNumber = 0;
    while (std::getline(input, line)) {
        ++lineNumber;
        if (line.empty()) {
            continue;
        }
        const auto separator = line.find('\t');
        if (separator == std::string::npos) {
            error = "Неверный формат строки " + std::to_string(lineNumber) +
                    " в файле участников.";
            return false;
        }

        int id = 0;
        std::istringstream idStream(line.substr(0, separator));
        if (!(idStream >> id) || id <= 0 || !idStream.eof()) {
            error = "Неверный идентификатор в строке " + std::to_string(lineNumber) + ".";
            return false;
        }
        const std::string name = line.substr(separator + 1);
        if (name.empty()) {
            error = "Пустое имя в строке " + std::to_string(lineNumber) + ".";
            return false;
        }
        participants.push_back({id, name, std::nullopt});
    }
    return true;
}

bool saveParticipants(const std::filesystem::path& path,
                      const std::vector<Participant>& participants,
                      std::string& error) {
    const auto temporary = path.string() + ".tmp";
    std::ofstream output(temporary, std::ios::trunc);
    if (!output) {
        error = "Не удалось создать временный файл участников.";
```

```
        return false;
    }
    for (const auto& participant : participants) {
        output << participant.id << '\t' << participant.name << '\n';
    }
    output.close();
    if (!output) {
        error = "Ошибка записи файла участников.";
        return false;
    }

    std::error_code code;
    std::filesystem::rename(temporary, path, code);
    if (code) {
        std::filesystem::remove(path, code);
        code.clear();
        std::filesystem::rename(temporary, path, code);
    }
    if (code) {
        error = "Не удалось заменить файл участников: " + code.message();
        std::filesystem::remove(temporary);
        return false;
    }
    return true;
}

bool saveResults(const std::filesystem::path& path,
                 const std::vector<Participant>& standings,
                 int prizePlaces,
                 std::string& error) {
    std::ofstream output(path, std::ios::trunc);
    if (!output) {
        error = "Не удалось создать файл результатов: " + path.string();
        return false;
    }

    const int cutoff = standings.at(static_cast<std::size_t>(prizePlaces - 1)).score.value();
    output << "ИТОГИ КОНКУРСА\n\n";
    output << "Место | ID | Имя | Балл | Статус\n";
    output << std::string(60, '-') << '\n';

    int place = 0;
    int previousScore = -1;
    for (std::size_t index = 0; index < standings.size(); ++index) {
        const int score = standings[index].score.value();
        if (score != previousScore) {
            place = static_cast<int>(index) + 1;
            previousScore = score;
        }
        output << place << " | " << standings[index].id << " | " << standings[index].name
               << " | " << score << " | " << (score >= cutoff ? "призёр" : "") << '\n';
    }
    return static_cast<bool>(output);
}

}  // namespace Storage


#include "Contest.h"
#include "Storage.h"

#include <filesystem>
#include <iostream>
#include <limits>
#include <locale>
#include <memory>
#include <string>
#include <vector>

namespace {

const std::filesystem::path ParticipantsFile = "participants.txt";
const std::filesystem::path ResultsFile = "results.txt";

void clearInput() {
    std::cin.clear();
```

```
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

int readInteger(const std::string& prompt, int minimum, int maximum) {
    while (true) {
        std::cout << prompt;
        int value = 0;
        if (std::cin >> value && value >= minimum && value <= maximum) {
            clearInput();
            return value;
        }
        std::cout << "Введите целое число от " << minimum << " до " << maximum << ".\n";
        clearInput();
    }
}

std::string readLine(const std::string& prompt) {
    std::cout << prompt;
    std::string value;
    std::getline(std::cin, value);
    return value;
}

void printParticipants(const std::vector<Participant>& participants) {
    if (participants.empty()) {
        std::cout << "Список участников пуст.\n";
        return;
    }
    std::cout << "ID | Имя\n";
    std::cout << std::string(50, '-') << '\n';
    for (const auto& participant : participants) {
        std::cout << participant.id << " | " << participant.name;
        if (participant.score.has_value()) {
            std::cout << " — " << participant.score.value() << " балл(ов)";
        }
        std::cout << '\n';
    }
}

void printStandings(const Contest& contest) {
    const auto standings = contest.standings();
    const int cutoff = standings.at(static_cast<std::size_t>(contest.prizePlaces() -
    ↪  1)).score.value();
    std::cout << "\nИТОГИ КОНКУРСА\n";
    std::cout << "Место | ID | Имя | Балл | Статус\n";
    std::cout << std::string(60, '-') << '\n';

    int place = 0;
    int previousScore = -1;
    for (std::size_t index = 0; index < standings.size(); ++index) {
        const int score = standings[index].score.value();
        if (score != previousScore) {
            place = static_cast<int>(index) + 1;
            previousScore = score;
        }
        std::cout << place << " | " << standings[index].id << " | " << standings[index].name
                  << " | " << score << " | " << (score >= cutoff ? "призёр" : "") << '\n';
    }
}

bool loadSavedParticipants(Contest& contest) {
    std::vector<Participant> saved;
    std::string error;
    if (!Storage::loadParticipants(ParticipantsFile, saved, error)) {
        std::cerr << "Ошибка: " << error << '\n';
        return false;
    }
    for (const auto& participant : saved) {
        if (!contest.registerParticipant(participant.id, participant.name, error)) {
            std::cerr << "Ошибка загрузки участника ID " << participant.id << ": " << error << '\n';
            return false;
        }
    }
    if (!saved.empty()) {
        std::cout << "Загружено участников: " << saved.size() << ".\n";
    }
```

```
    return true;
}

bool saveRegistration(const Contest& contest) {
    std::string error;
    if (!Storage::saveParticipants(ParticipantsFile, contest.participants(), error)) {
        std::cerr << "Ошибка: " << error << '\n';
        return false;
    }
    std::cout << "Регистрация сохранена в " << ParticipantsFile.string() << ".\n";
    return true;
}

void registrationMenu(Contest& contest) {
    while (contest.state() == Contest::State::Registration) {
        std::cout << "\n1. Зарегистрировать участника\n"
                     "2. Показать участников\n"
                     "3. Завершить регистрацию\n"
                     "0. Сохранить и выйти\n";
        const int choice = readInteger("Выберите действие: ", 0, 3);
        if (choice == 0) {
            saveRegistration(contest);
            return;
        }
        if (choice == 1) {
            const int id = readInteger("Идентификатор: ", 1, std::numeric_limits<int>::max());
            const std::string name = readLine("Имя участника: ");
            std::string error;
            if (contest.registerParticipant(id, name, error)) {
                std::cout << "Участник зарегистрирован.\n";
            } else {
                std::cout << "Ошибка: " << error << '\n';
            }
        } else if (choice == 2) {
            printParticipants(contest.participants());
        } else {
            std::string error;
            if (contest.finishRegistration(error)) {
                std::error_code code;
                std::filesystem::remove(ParticipantsFile, code);
                std::cout << "Регистрация завершена. Переход к вводу баллов.\n";
            } else {
                std::cout << "Ошибка: " << error << '\n';
            }
        }
    }
}

void scoringMenu(Contest& contest) {
    while (contest.state() == Contest::State::Scoring) {
        std::cout << "\n1. Зарегистрировать балл\n"
                     "2. Показать участников и баллы\n"
                     "3. Завершить конкурс\n";
        const int choice = readInteger("Выберите действие: ", 1, 3);
        if (choice == 1) {
            const int id = readInteger("Идентификатор участника: ", 1,
                                       std::numeric_limits<int>::max());
            const int score = readInteger("Балл: ", 0, 100);
            std::string error;
            if (contest.registerScore(id, score, error)) {
                std::cout << "Балл зарегистрирован.\n";
            } else {
                std::cout << "Ошибка: " << error << '\n';
            }
        } else if (choice == 2) {
            printParticipants(contest.participants());
        } else {
            std::string error;
            if (contest.finishContest(error)) {
                std::cout << "Конкурс завершён.\n";
            } else {
                std::cout << "Ошибка: " << error << '\n';
            }
        }
    }
}
```

```
}  // namespace

int main() {
    try {
        std::locale::global(std::locale(""));
    } catch (const std::exception&) {
    }

    std::cout << "КОНКУРС\n";
    const int prizePlaces = readInteger("Количество призовых мест (1–10): ", 1, 10);
    Contest contest(prizePlaces);
    if (!loadSavedParticipants(contest)) {
        return 1;
    }

    registrationMenu(contest);
    if (contest.state() == Contest::State::Registration) {
        return 0;
    }

    scoringMenu(contest);
    printStandings(contest);

    std::string error;
    if (!Storage::saveResults(ResultsFile, contest.standings(), contest.prizePlaces(), error)) {
        std::cerr << "Ошибка: " << error << '\n';
        return 1;
    }
    std::cout << "\nРезультаты сохранены в " << ResultsFile.string() << ".\n";
    return 0;
}
```

```
#include "Contest.h"

#include <cassert>
#include <iostream>
#include <string>

namespace {

void add(Contest& contest, int id, const std::string& name) {
    std::string error;
    assert(contest.registerParticipant(id, name, error));
}

void score(Contest& contest, int id, int value) {
    std::string error;
    assert(contest.registerScore(id, value, error));
}

void testTieExpandsWinners() {
    Contest contest(2);
    add(contest, 1, "Анна");
    add(contest, 2, "Борис");
    add(contest, 3, "Вера");
    add(contest, 4, "Глеб");

    std::string error;
    assert(contest.finishRegistration(error));
    score(contest, 1, 100);
    score(contest, 2, 90);
    score(contest, 3, 90);
    score(contest, 4, 70);
    assert(contest.finishContest(error));

    const auto winners = contest.winners();
    assert(winners.size() == 3);
    assert(winners[0].id == 1);
    assert(winners[1].score == 90);
    assert(winners[2].score == 90);
}

void testValidation() {
    Contest contest(1);
```

```
    std::string error;
    assert(!contest.registerParticipant(0, "Анна", error));
    assert(contest.registerParticipant(1, "Анна", error));
    assert(!contest.registerParticipant(1, "Другое имя", error));
    assert(contest.finishRegistration(error));
    assert(!contest.registerScore(1, 101, error));
    assert(contest.registerScore(1, 50, error));
    assert(!contest.registerScore(1, 60, error));
    assert(contest.finishContest(error));
}

void testCannotFinishWithoutAllScores() {
    Contest contest(1);
    add(contest, 1, "Анна");
    add(contest, 2, "Борис");
    std::string error;
    assert(contest.finishRegistration(error));
    score(contest, 1, 100);
    assert(!contest.finishContest(error));
}

}  // namespace

int main() {
    testTieExpandsWinners();
    testValidation();
    testCannotFinishWithoutAllScores();
    std::cout << "Все тесты пройдены.\n";
    return 0;
}
```

```
cmake_minimum_required(VERSION 3.16)

project(ContestCourseWork VERSION 1.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_library(contest_core
    src/Contest.cpp
    src/Storage.cpp
)

target_include_directories(contest_core PUBLIC include)

if(MSVC)
    target_compile_options(contest_core PRIVATE /W4 /permissive- /utf-8)
else()
    target_compile_options(contest_core PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(contest src/main.cpp)
target_link_libraries(contest PRIVATE contest_core)

if(MSVC)
    target_compile_options(contest PRIVATE /W4 /permissive- /utf-8)
else()
    target_compile_options(contest PRIVATE -Wall -Wextra -Wpedantic)
endif()

include(CTest)
if(BUILD_TESTING)
    add_executable(contest_tests tests/contest_tests.cpp)
    target_link_libraries(contest_tests PRIVATE contest_core)
    add_test(NAME contest_tests COMMAND contest_tests)
endif()
```